

Análisis del Dynamic Array y Snake Challenge

Juan Camilo Gómez Chacón, Pablo Alejandro Villa Gacha y Santiago Cortés Espinosa
{juagomezch, pvilla, scortese}@unal.edu.co

27 de septiembre de 2026

Pregunta 1: Análisis de DynamicArray

1. Definición de Invariantes de DynamicArray

Para asegurarnos de que la estructura funcione bien y no tengamos problemas de memoria con la clase `DynamicArray`, mantenemos estas tres reglas o invariantes durante toda la ejecución:

- **Invariante de Tamaño y Capacidad:**

$$0 \leq \text{size} \leq \text{capacity} \quad \text{con} \quad \text{capacity} \geq 1$$

Sabemos que la cantidad de elementos guardados (**size**) nunca puede ser negativa ni sobrepasar la capacidad física del arreglo interno. Además, la capacidad siempre tiene que ser estrictamente positiva.

- **Invariante de Contigüidad y Accesibilidad:**

$$\forall i \in \mathbb{Z}, \quad 0 \leq i < \text{size} \implies \text{array}[i] \neq \text{null}$$

Los datos válidos siempre van a ocupar posiciones consecutivas comenzando desde el índice 0 hasta $\text{size} - 1$.

- **Invariante de Limpieza de Referencias:**

$$\forall i \in \mathbb{Z}, \quad \text{size} \leq i < \text{capacity} \implies \text{array}[i] = \text{null}$$

Cualquier casilla del arreglo que esté por encima del tamaño lógico se deja explícitamente en `null`. Esto se hace para que el Garbage Collector pueda liberar los objetos que ya no estamos usando y evitar fugas de memoria.

2. Complejidad Worst-Case de los Métodos

- `get(int index)` $\rightarrow O(1)$: Primero valida que $0 \leq \text{index} < \text{size}$ en tiempo constante $O(1)$ y luego accede directo a la dirección de memoria usando la fórmula de desplazamiento base.
- `set(int index, T value)` $\rightarrow O(1)$: Hace la misma revisión de límites y sobrescribe el valor en la posición indicada mediante acceso indexado directo en $O(1)$.
- `removeLast()` $\rightarrow O(1)$: Simplemente decrementa `size`, le asigna `null` a la posición `array[size]` para evitar referencias colgadas y retorna el elemento borrado. Ninguna de estas cosas depende de n .
- `append(T value)` $\rightarrow O(n)$: Si todavía hay espacio libre, mete el elemento al final en $O(1)$. Sin embargo, en el peor caso (cuando `size = capacity`), toca duplicar el tamaño del arreglo, crear una estructura nueva y copiar los n elementos uno por uno, lo que termina costando $O(n)$.

3. Justificación del Costo Worst-Case vs. Costo Amortizado

Aquí comparamos dos perspectivas distintas:

1. **Perspectivas diferentes:** El peor caso $O(n)$ mide lo que cuesta una operación aislada cuando justo le toca hacer el `resize`. En cambio, el costo amortizado $O(1)$ calcula el promedio por operación a lo largo de una secuencia de n inserciones seguidas desde que la estructura está vacía.
2. **Mecanismo de compensación:** Para llegar a una capacidad de n elementos duplicando el tamaño, el arreglo tuvo que pasar por capacidades $1, 2, 4, 8, \dots, n$. El total de elementos copiados en todo ese proceso equivale a la suma de una serie geométrica:

$$T_{\text{copias}} = \sum_{i=0}^{\log_2 n - 1} 2^i = 2^{\log_2 n} - 1 = n - 1$$

Si le sumamos las n inserciones baratas de $O(1)$, el trabajo total nos queda:

$$\text{Costo acumulado} = n + (n - 1) = 2n - 1 < 2n$$

3. **Conclusión:** Como hacer las n operaciones cuesta en total menos de $2n$, si dividimos ese costo acumulado entre n nos da un promedio estricto de $\frac{2n}{n} = O(1)$ amortizado por cada `append`.

Pregunta 2: Snake: Una implementación aparentemente eficiente

1. Complejidad temporal de REMOVE_TAIL()

Revisando el código propuesto, la función para eliminar la cola hace un bucle `for` desde $i = 1$ hasta $\text{size} - 1$:

$$\sum_{i=1}^{\text{size}-1} 1 = \text{size} - 1$$

Cada iteración hace una asignación de memoria más la reducción de `size`. Por ende, para una serpiente de longitud k , ejecutar `REMOVE_TAIL()` cuesta $O(k)$ operaciones.

2. Análisis de complejidad de m movimientos

En cada movimiento sin comer, la serpiente ejecuta dos cosas principales:

- Insertar la nueva cabeza con `snake.append(newHead)`, que cuesta $O(1)$ en la mayoría de pasos.
- Eliminar la cola anterior con `REMOVE_TAIL()`, que como vimos arriba cuesta $O(k)$ en cada movimiento.

Como la serpiente hace m movimientos manteniendo su longitud k constante, el costo total de la secuencia viene siendo $O(m \cdot k)$.

3. Determinación de validez de la afirmación

La afirmación es **falsa**. El argumento de decir Como `append()` es $O(1)$ amortizado, entonces `MOVE()` también es $O(1)$ amortizado ignora por completo que `REMOVE_TAIL()` se ejecuta en **cada uno** de los movimientos. Como eliminar la cola obliga a correr todos los elementos a la izquierda con un costo de $O(k)$ por paso, el costo por movimiento es $O(k)$ y no $O(1)$.

4. Identificación del bottleneck

El cuello de botella de esta representación es claramente la función `REMOVE_TAIL()`. Mover manualmente todos los elementos del arreglo hacia la izquierda en cada paso genera un trabajo de $O(k)$ inútil que destruye la eficiencia cuando la serpiente se vuelve grande.

Pregunta 3: Redesign Challenge

1. Variables de estado adicionales

Para arreglar este problema sin mover elementos, usamos la idea de un **búfer circular**. Definimos dos variables adicionales:

- **head**: Guarda la posición física en el arreglo donde está la cabeza actual.
- **tail**: Guarda la posición física donde se encuentra la cola.

También podemos usar **head** y **size**, calculando la cola según se necesite.

2. Transformación de un índice lógico i

Siguiendo la convención del quiz donde el índice lógico 0 es la cola y el índice $k - 1$ es la cabeza, la fórmula para transformar cualquier índice lógico i ($0 \leq i < \text{size}$) a su posición física real en el arreglo es:

$$\text{posición_física}(i) = (\text{tail} + i) \quad (\text{mód capacity})$$

Gracias al operador módulo (mód capacity) , nos aseguramos de que el índice siempre gire dentro de los límites del arreglo sin tener que desplazar datos.

Pregunta 4: Análisis Amortizado

a) Método Agregado

Sabemos que la serpiente arranca con longitud 2 y también sabemos que en cada paso hace `addHead` antes de `removeTail`. Entonces sabemos que la longitud máxima en cualquier instante es $n_{\text{máx}} = g + 3$, donde la g representa la cantidad de manzanas consumidas.

Entonces, la capacidad arranca en 2 y se va duplicando en cada redimensionamiento $(2, 4, 8, \dots, 2^k)$. El total de los elementos copiados equivale a la suma de todas las capacidades anteriores lo que se ve en esta ecuación:

$$T_{\text{copias}} = \sum_{j=1}^k 2^j = 2^{k+1} - 2$$

Como el último resize se dio porque $2^k < g + 3$, se cumple que $2^{k+1} < 2g + 6$ lo que nos lleva a que:

$$T_{\text{copias}} < 2g + 6 = O(g)$$

Ahí vemos que el total de las copias al final depende solo del número de manzanas g .

b) Demostración del costo lineal

Ahora un redimensionamiento pasa justo cuando el arreglo está lleno con capacidad 2^i e intentamos meter un elemento nuevo. En ese momento nos toca copiar esos 2^i elementos.

Para llegar a una longitud n , sabemos que el total de copias equivale a la suma de las capacidades anteriores:

$$T(n) = \sum_{i=1}^k 2^i = 2^{k+1} - 2$$

Como el último resize se activó porque $2^k < n$, sabemos que $2^{k+1} < 2n$. Y si reemplazamos esto nos queda:

$$T(n) < 2n - 2 < 2n = O(n)$$

Con esto ya demostramos que la suma de copias siempre está acotada por $2n$, lo que nos da un costo lineal $O(n)$ respecto a los elementos guardados.

c) Costo total y costo amortizado

Si sumamos los costos de los m movimientos que hace la serpiente:

- m escrituras de la nueva cabeza: m .
- $(m - g)$ eliminaciones de cola: $O(m - g)$.
- Copias hechas en los redimensionamientos: $O(g)$.

$$\text{Costo Total} = m + O(m - g) + O(g) = O(m)$$

Y si dividimos este costo total entre los m movimientos que se hicieron:

$$\hat{c}_{\text{addHead}} = \frac{O(m)}{m} = O(1)$$

d) Papel de m y g

- m es el numero de movimientos totales. Básicamente aporta el trabajo constante de $O(1)$ por cada paso sencillo.
- g son las manzanas que se comió. Es lo único que hace crecer la serpiente de verdad y dispara los resizes de costo $O(n)$.

e) Método del Potencial

Definimos la función potencial Φ en el estado i de la siguiente manera:

$$\Phi_i = 2 \cdot \text{size}_i - \text{capacity}_i$$

Al principio tenemos $\text{size} = 2$ y $\text{capacity} = 2$, así que $\Phi_0 = 2(2) - 2 = 2 \geq 0$. Después de meter un `resize` con tamaño $C+1$ y capacidad $2C$, el potencial nos queda $\Phi = 2(C+1) - 2C = 2 \geq 0$. Por ende, sabemos que siempre se va a mantener $\Phi_i \geq 0$.

- **Movimiento normal (sin comer):** El tamaño no cambia ($\Delta \text{size} = 0$) y la capacidad tampoco ($\Delta \text{capacity} = 0$), entonces $\Delta \Phi = 0$.

$$\hat{c}_i = c_i + \Delta \Phi = 1 + 0 = O(1)$$

- **Movimiento con `resize` ($\text{size} = C$):** El costo real aquí es $c_i = C + 1$ (las copias mas la escritura). El potencial inicial era $\Phi_{\text{inicio}} = C$ y al final queda $\Phi_{\text{fin}} = 2$, así que el cambio es $\Delta \Phi = 2 - C$.

$$\hat{c}_i = (C + 1) + (2 - C) = 3 = O(1)$$

Cuando ejecutamos `removeTail()` se elimina un elemento y se reduce el tamaño, pero la capacidad se queda igual. Eso hace que baje el potencial, pero solo estamos consumiendo créditos que ya habíamos acumulado antes, manteniendo $\Phi_i \geq 0$ sin tirarnos los resizes futuros.

Pregunta 5: The Adversarial Snake

a) Poder del adversario

Si asumimos por ejemplo que la serpiente no se reduce, el momento en que salen las manzanas no cambia para nada la cantidad total de copias que tocan hacer para llegar al tamaño n . Al final las capacidades por las que pasa el arreglo son exactamente las mismas.

Para que un adversario pudiera obligarnos a hacer mas copias, la estructura tendría que tener una contracción automatica (por ejemplo reduciendo la capacidad a la mitad cuando baje al 50 %). Ahí si podría alternar entre comer y no comer justo en el limite de la capacidad para forzar resizes a cada rato.

b, c, d, e, f) Comparación de políticas

Política	# Resizes	Total copias	Costo amortizado	Espacio sin usar
A ($C + 1$)	$O(n)$	$O(n^2)$	$O(n)$	1 elemento ($\approx 0\%$)
B ($2C$)	$O(\log_2 n)$	$< 2n = O(n)$	$O(1)$	Hasta $n - 1$ ($\approx 50\%$)
C ($1,5C$)	$O(\log_{1,5} n)$	$\leq 3n = O(n)$	$O(1)$	Hasta $0,5C$ ($\approx 33,3\%$)

Demostración para la familia $C_{\text{new}} = \lceil \alpha C \rceil$

Sabemos que no hace falta duplicar la capacidad ($\alpha = 2$), sino que nos sirve cualquier constante $\alpha > 1$.

Si la capacidad va creciendo como $C_k = C_0 \cdot \alpha^k$, para llegar a un tamaño n con K redimensionamientos se cumple que $\alpha^{K-1} < \frac{n}{C_0}$.

Si sumamos las copias totales nos queda:

$$T(n) = C_0 \sum_{k=0}^{K-1} \alpha^k = C_0 \left(\frac{\alpha^K - 1}{\alpha - 1} \right) < \frac{\alpha n - C_0}{\alpha - 1} = O(n)$$

Entonces el costo amortizado por crecimiento viene siendo:

$$\hat{c} = \frac{T(n)}{n} < \frac{\alpha}{\alpha - 1} = O(1)$$

Como se ve, esta es una serie geometrica que converge para cualquier $\alpha > 1$, se demuestra que la familia $C_{\text{new}} = \lceil \alpha C \rceil$ siempre mantiene un costo amortizado constante $O(1)$. Ahora, si tuvieramos $\alpha = 1$ (como en la Política A), eso se convierte en una serie aritmetica dando un costo de $O(n^2)$ y un costo amortizado de $O(n)$.